Resolução da questão 2

1. Formato de Ponto Fixo (7 bits inteiros + 4 bits fracionários)

O formato possui 11 bits no total (7 inteiros com sinal + 4 fracionários):

- **Parte inteira (7 bits com sinal):** $b_6\ b_5\ b_4\ b_3\ b_2\ b_1\ b_0$, onde b_6 é o bit de sinal com peso $-2^6 = -64$.
- **Parte fracionária (4 bits):** $b_{-1}\ b_{-2}\ b_{-3}\ b_{-4}$, com pesos $2^{-1}, 2^{-2}, 2^{-3}, 2^{-4}$.

Bit	b_6 (sinal)	b_5	b_4	b_3	b_2	b_1	b_0	.	b_{-1}	b_{-2}	b_{-3}	b_{-4}
Peso	−64	+32	+16	+8	+4	+2	+1	.	0,5	0,25	0,125	0,0625

Faixa de representação do formato: de **−64,0** até **+63,9375**.

2. Conversão dos Números para Binário

2.1. Conversão de B = +10,25

Como B é positivo, o bit de sinal é 0.

- **Parte inteira (10_{10})** por divisões sucessivas por 2: 10 / 2 = 5 (resto **0**); 5 / 2 = 2 (resto **1**); 2 / 2 = 1 (resto **0**); 1 / 2 = 0 (resto **1**).
  Preenchendo 7 bits: $\mathbf{10_{10} = 0001010_2}$.
- **Parte fracionária ($0{,}25_{10}$)** por multiplicações sucessivas por 2: 0,25 × 2 = 0,50 (parte inteira: **0**); 0,50 × 2 = 1,00 (parte inteira: **1**, resto 0).
  Parte fracionária em 4 bits: $\mathbf{.0100_2}$.

$$\boldsymbol{B = 0001010.0100_2\ (+10{,}25_{10})}$$

2.2. Conversão de A = −27,30

Como A é negativo, primeiro calculamos a magnitude positiva (+27,30) e depois aplicamos o complemento de 2.

- **Parte inteira (27_{10})** por divisões sucessivas por 2: 27 / 2 = 13 (resto **1**); 13 / 2 = 6 (resto **1**); 6 / 2 = 3 (resto **0**); 3 / 2 = 1 (resto **1**); 1 / 2 = 0 (resto **1**).
  Preenchendo 7 bits: $\mathbf{27_{10} = 0011011_2}$.
- **Parte fracionária ($0{,}30_{10}$)** por multiplicações sucessivas por 2: 0,30 × 2 = 0,60 (**0**); 0,60 × 2 = 1,20 (**1**); 0,20 × 2 = 0,40 (**0**); 0,40 × 2 = 0,80 (**0**).
  (a próxima multiplicação seria 0,80 × 2 = 1,60, gerando dígito 1 e a dízima periódica $0{,}0100110011\ldots_2$).
  Truncando para 4 bits fracionários: $\mathbf{0{,}30_{10} \approx .0100_2\ (= 0{,}25_{10})}$.
  Magnitude positiva truncada: $\mathbf{|A| = 0011011.0100_2\ (+27{,}25_{10})}$.
- **Complemento de 2 de A:** Invertemos os bits (`1100100.1011`) e somamos 1 no LSB:

```
  1100100.1011
+ 0000000.0001
--------------
  1100100.1100
```

  Conferência em decimal: −64 + 32 + 4 + 0,5 + 0,25 = $\mathbf{-27{,}25_{10}}$.

$$\boldsymbol{A = 1100100.1100_2\ (-27{,}25_{10})}$$

3. Subtração D = A − B em Binário

Em complemento de 2, calculamos a subtração somando A com o oposto de B: **D = A + (−B)**.

3.1. Calculando −B:

Invertendo os bits de B (`1110101.1011`) e somando 1 no LSB:

$$-B = 1110101.1100_2\ (-10{,}25_{10})$$

3.2. Operação armada com todos os vem-uns:

```
vem-uns:   1 1 0 0 1 0 0 . 1 1 0 0
     A:      1 1 0 0 1 0 0 . 1 1 0 0    (-27,25)
  (-B):    + 1 1 1 0 1 0 1 . 1 1 0 0    (-10,25)
           ------------------------
     D:    (1) 0 1 1 0 1 0 . 1 0 0 0    (-37,50)
```

(O vem-um que sai do bit de sinal, carry de saída = 1, é descartado na aritmética de 11 bits).

- **Operando A:** `1100100.1100` ($-27{,}25_{10}$)
- **Operando −B:** `1110101.1100` ($-10{,}25_{10}$)
- **Vetor de vem-uns:** `1 1 0 0 1 0 0 . 1 1 0 0` (gerados nas colunas de peso 0,25, 0,5, 1, 4, 32 e no bit de sinal).
- **Resultado D:** `1011010.1000` ($-64 + 16 + 8 + 2 + 0{,}5 = \mathbf{-37{,}50_{10}}$).

4. Análise de Overflow na Subtração

Não houve overflow, pelos seguintes motivos:

- **Sinais dos operandos:** Somamos dois números negativos (A = −27,25 e −B = −10,25) e o resultado obtido também é negativo (D = −37,50, bit de sinal 1). Em complemento de 2 só ocorreria overflow se a soma de dois negativos resultasse em positivo.
- **Carries no bit de sinal (critério da ULA):** O carry que entra no bit de sinal ($c_{in} = 1$) é igual ao carry que sai dele ($c_{out} = 1$). O indicador de overflow é $V = c_{in} \oplus c_{out} = 1 \oplus 1 = \mathbf{0}$.
- **Faixa de representação:** O resultado −37,50 cabe perfeitamente dentro do limite do formato (−64,0 a +63,9375).

5. Soma E = D + B em Binário

Somamos o resultado obtido D (−37,50) com o operando B (+10,25):

5.1. Operação armada com todos os vem-uns:

```
vem-uns:   0 0 1 1 0 1 0 . 0 0 0 0
     D:      1 0 1 1 0 1 0 . 1 0 0 0    (-37,50)
     B:    + 0 0 0 1 0 1 0 . 0 1 0 0    (+10,25)
           ------------------------
     E:      1 1 0 0 1 0 0 . 1 1 0 0    (-27,25)
```

- **Vetor de vem-uns:** `0 0 1 1 0 1 0 . 0 0 0 0` (carries gerados nas colunas de peso 2 e 8, propagando na coluna de peso 16).
- **Resultado E:** `1100100.1100` ($-64 + 32 + 4 + 0{,}5 + 0{,}25 = \mathbf{-27{,}25_{10}}$).

6. Valor de E, Comparação com A e Comentários

- **Valor de E:** Em binário é **`1100100.1100`**, que equivale a **−27,25** em decimal.
- **Este valor é igual a A?**
  • **No valor de máquina (binário): Sim.** Em binário, E é exatamente idêntico ao valor de A armazenado no formato com 11 bits (`1100100.1100`).
  • **Em relação ao valor decimal original: Não.** O valor original era −27,30 e obtivemos −27,25, com uma diferença de 0,05.
- **Comentários:**
  Como não houve overflow na subtração nem na soma, a operação é reversível em complemento de 2: E = D + B = (A − B) + B = A. Por isso a representação binária guardada na máquina foi recuperada com exatidão.
  A diferença de 0,05 em relação ao decimal original ocorreu porque a fração $0{,}30_{10}$ não tem representação binária exata finita (gera dízima periódica). Como o formato possui apenas 4 bits fracionários, o número precisou ser truncado em $.0100_2 = 0{,}25_{10}$, introduzindo esse erro de truncamento de 0,05 logo na conversão inicial.